



第十章spark

🕒 Created @2025年6月12日 10:10

关键知识点

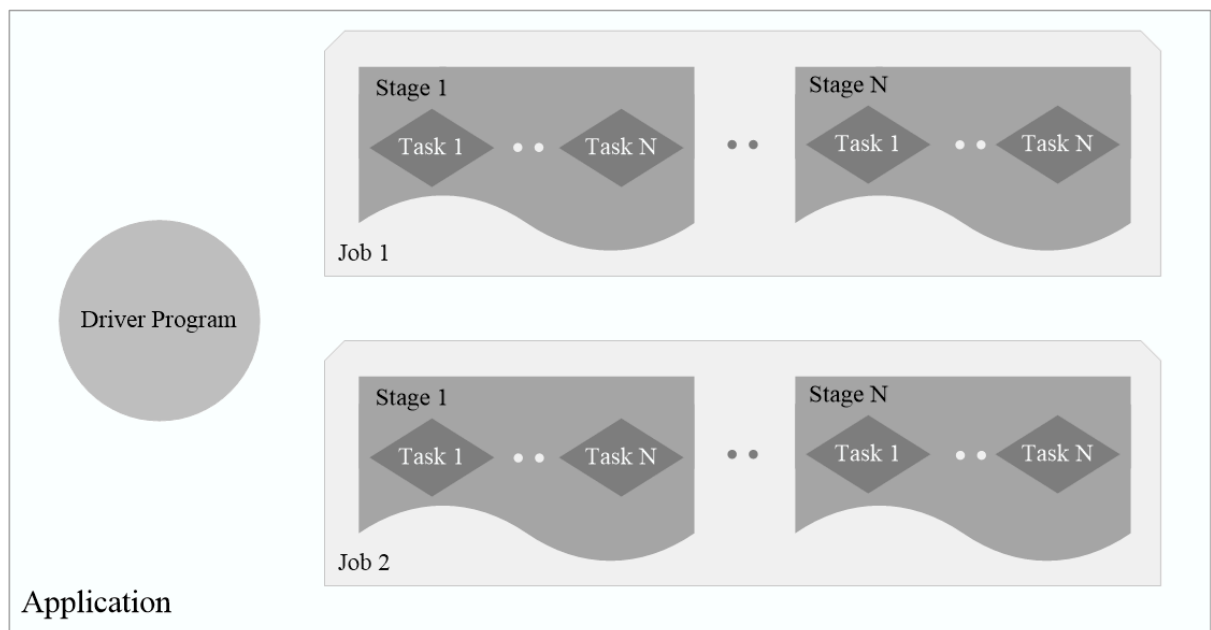
1. **Spark** 是基于**内存计算**的大数据并行计算框架
2. **Spark** 的主要特点：运行速度快、容易使用、通用性、运行模式多样
3. **Spark** 使用先进的**有向无环图 DAG** 执行引擎，以支持**循环数据流与内存计算**
4. **Scala** 是一门**多范式编程语言**，平滑地集成了**面向对象和函数式语言**的特性
5. **Spark** 的五种部署模式
 - a. **Local**、**Standalone**、**Spark on Mesos**、**Spark on YARN**、**Spark on Kubernetes**
6. **Scala** 交互模式 **REPL** 是一个交互式解释器，可以即时编译、运行代码并返回结果
 - a. REPL：R(read)、E(evaluate)、P(print)、L(loop)
7. **RDD** 的 **Transformation** 分为两种：**窄依赖**（**map**、**filter**）、**宽依赖**（**reduceByKey**）
 - a. 窄依赖与宽依赖的主要区别在于是否包含 **Shuffle** 操作
 - i. **宽依赖包含 Shuffle 操作**
 - b. 窄依赖：一个或多个父 **RDD** 的一个分区对应一个子 **RDD** 的一个分区
 - i. 父 **RDD** 的一个分区只被一个子 **RDD** 的一个分区使用
 - c. 宽依赖：一个父 **RDD** 的一个分区对应一个子 **RDD** 的多个分区
8. **Shuffle**：将 **Map** 输出的中间结果分发到 **Reduce** 任务所在的机器（考！！）
 - a. **Shuffle** 会产生大量的网络数据分发，带来昂贵的**网络传输开销**与**磁盘IO开销**
9. 当 **Spark** 应用被提交时，由 **Driver** 创建一个 **SparkContext**（考！！）
10. **RDD** 提供的转换接口适用于**粗粒度**的数据转换操作，而不是针对某个数据项的细粒度修改（考）

Spark生态系统

Spark生态系统组件	其他框架	应用场景	时间框架
Spark Core	MapReduce 、 Hive	复杂的批量数据处理	小时级别
Spark SQL	Dremel 、 Impala	基于历史数据的交互 式查询	分钟级、秒级
Spark Streaming 、 Structured Streaming	S4 、 Storm	基于实时数据流的数据 处理	毫秒级、秒级
MLlib	Mahout	基于历史数据的数据 挖掘	——
GraphX	Pregel	图结构数据的处理	——

Spark运行架构

基本概念

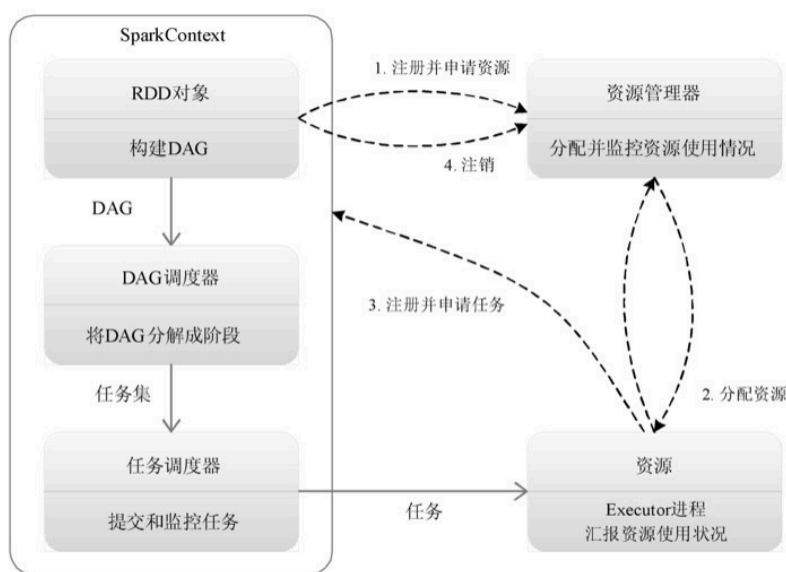


- **RDD**：分布式内存的一个抽象概念，提供一种高度受限的共享内存模型
 - 本质是可并行处理的只读数据集（分布式数据集）
 - **RDD** 只能基于稳定的物理存储中的数据集创建或在其他 **RDD** 上执行确定的转换操作创建
- **DAG**（有向无环图）：反映 **RDD** 之间的依赖关系
- **Executor**：工作进程，负责在 Spark 作业 Job 中运行任务 **Task**

- 应用 **Application**：用户编写的 **Spark** 应用程序
- 作业 **Job**：一个作业 **Job** 包含多个 **RDD** 及作用于相应 **RDD** 上的各种操作
- 阶段 **Stage**：作业 **Job** 的基本调度单位
 - 一个作业 **Job** 分为多组任务 **Task**，每组任务称为阶段或任务集
- 任务 **Task**：运行在 **Executor** 上的工作单元

spark运行基本流程

1. Driver创建 **SparkContext**，构建应用运行环境，负责资源申请、任务分配和监控。
2. 资源管理器为 **Executor** 分配资源并启动其进程。
3. **SparkContext** 根据 **RDD** 依赖关系构建DAG图，通过 **DAGScheduler** 将其解析为 **Stage**，生成TaskSet并交给 **TaskScheduler**；**Executor** 申请 **Task**，**TaskScheduler** 分发 **Task** 并提供程序代码。
4. **Task** 在 **Executor** 上运行，将结果反馈给 **TaskScheduler** 和 **DAGScheduler**，运行完成后写入数据并释放资源。



Spark中各种概念之间的相互关系

RDD操作（数据运算）

- **RDD** 提供**行动 Action** 和**转换 Transformation** 两种类型的操作
 - **Action** 用于执行计算并指定输出的形式

- **Transformation** 用于指定 RDD 之间的依赖关系
- 主要区别：Action接受RDD返回非RDD（输出一个值/结果），Transformation接受RDD返回RDD
- **RDD** 采用惰性调用，真正的计算发生在RDD的 **Action** 操作（考！！）
 - 对于 **Transformation** 操作，**Spark** 只记录其应用的基础数据集和相互之间的依赖关系
- 常用的Action操作API

Action API	说明
<code>count()</code>	返回数据集中的 元素个数
<code>collect()</code>	以 数组 的形式返回数据集中的 所有元素
<code>first()</code>	返回数据集中的 第一个元素
<code>take(n)</code>	以 数组 的形式返回数据集中的 前n个元素
<code>reduce(func)</code>	通过函数func（输入两个参数并返回一个值） 聚合 数据集中的元素
<code>foreach(func)</code>	将数据集中的 每个元素 传递到函数func中运行

- 常用的Transformation操作API

Transformation API	说明
<code>filter(func)</code>	筛选出 满足函数func的元素 ，并返回一个新的数据集
<code>map(func)</code>	将 每个元素 传递到func中，并将结果返回为一个新的数据集
<code>flatMap(func)</code>	与map()相似，但 每个输入元素 都可以映射到 零个或多个 输出结果
<code>groupByKey()</code>	应用于<K, V>键值对的数据集，返回一个新的<K, Iterable<V>>形式的数据集
<code>reduceByKey(func)</code>	应用于<K, V>键值对的数据集，返回一个新的<K, V>形式的数据集，其中的每个值是将每个key传递到函数func中进行 聚合 的结果

选择/填空

1. **RDD** 操作分为转换和行动两种类型，其中 `count` 属于行动类型的操作

判断

1. RDD 提供的转换接口既适用 filter 等粗粒度的转换，也适合某一数据项的细粒度转换 ✗
2. 在选择 Spark Streaming 和 Storm 时，对实时性要求高的企业更倾向于选择流计算框架 Storm ✓

简答

1. Spark 与 Hadoop 的对比
 - a. Hadoop 的缺陷：表达能力有限、磁盘I/O开销大、延迟高
 - i. 计算必须转化为 Map 和 Reduce 两个操作，难以描述复杂的数据处理过程
 - ii. 执行时需要从磁盘读取数据，计算完成后需要将中间结果写入磁盘，IO开销较大
 - iii. 计算可能需要分解为一系列按顺序执行的 MapReduce 任务，任务衔接会产生较高延迟
 - b. Spark 的优点
 - i. Spark 不局限于 Map 和 Reduce 操作，还提供多种数据集操作类型，编程模式更灵活
 - ii. Spark 提供内存计算，中间结果直接放置于内存，迭代运算效率更高
 - iii. Spark 基于 DAG 的任务调度执行机制，优于 MapReduce 的迭代执行机制
2. Spark 中各种概念（基本概念）之间的相互关系
 - a. 一个 Application 由一个 Driver 和若干个 Job 构成
 - b. 一个 Job 由多个 Stage 组成
 - c. 一个 Stage 由多个没有 Shuffle 关系的 Task 组成
3. RDD 在 Spark 中的运行过程（考！！）
 - a. 创建 RDD 对象
 - b. SparkContext 负责计算 RDD 之间的依赖关系，构建 DAG
 - c. DAGScheduler 负责把 DAG 分解成多个阶段，每个阶段包含多个任务
 - i. 每个任务会被任务调度器分发到各个工作节点上的 Executor 执行

代码/命令

- 读取一个 HDFS 文件，计数其中“Hello World”的行数

```

import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark.SparkConf

object HelloWorld {
  def main(args: Array[String]) { // Spark 应用程序的入口点
    val conf = new SparkConf().setAppName("Hello World").setMaster('
    val sc = new SparkContext(conf)
    val fileRDD = sc.textfile("hdfs://localhost:9000/examplefile") // 从 H
    val filterRDD = fileRDD.filter(_.contains("Hello World")) // 筛选 fileRDI
    filterRDD.cache()
    filterRDD.count() // 行动操作
  }
}

```

- 统计文本文件中包含字母 **a** 和字母 **b** 的行数

```

import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark.SparkConf

object SimpleApp {
  def main(args: Array[String]) {
    val logFile = "file:///usr/local/spark/README.md"
    val conf = new SparkConf().setAppName("Simple Application")
    val sc = new SparkContext(conf)
    val logData = sc.textFile(logFile, 2).cache()
    val numAs = logData.filter(_.contains("a")).count()
    val numBs = logData.filter(_.contains("b")).count()
    println("Lines with a: %s, Lines with b: %s".format(numAs, numBs))
  }
}

```